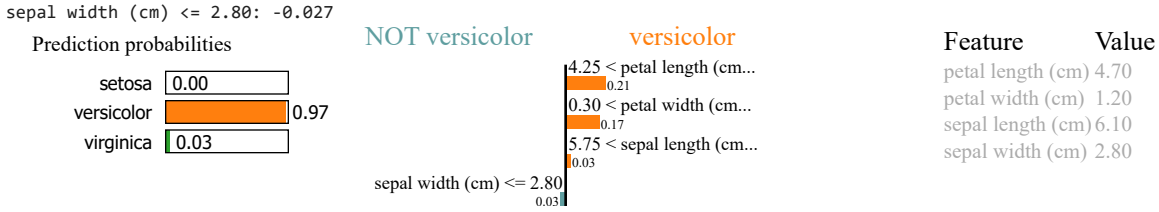
```
!pip install lime scikit-learn scikit-image matplotlib tensorflow
```

Show hidden output

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from lime.lime_tabular import LimeTabularExplainer

# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train model
model = RandomForestClassifier()
model.fit(X_train, y_train)

# Create LIME explainer
explainer = LimeTabularExplainer(
    X_train,
    feature_names=iris.feature_names,
    class_names=iris.target_names,
    mode='classification'
)

# Select a sample
i = 0

# Explain prediction
exp = explainer.explain_instance(
    X_test[i],
    model.predict_proba
)

# Print explanation
print("Explanation:")
for feature, weight in exp.as_list():
    print(f"{feature}: {weight:.3f}")

# Show visualization
exp.show_in_notebook(show_table=True)
```

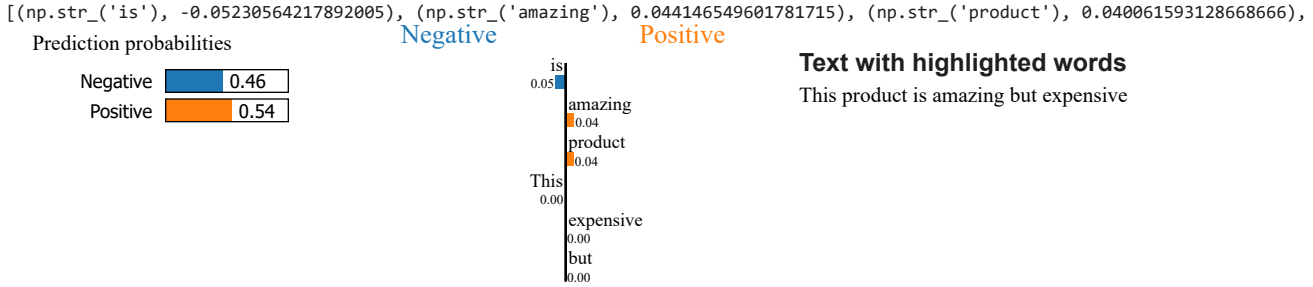
```
Explanation:
4.25 < petal length (cm) <= 5.10: 0.208
0.30 < petal width (cm) <= 1.30: 0.174
5.75 < sepal length (cm) <= 6.40: 0.027
sepal width (cm) <= 2.80: -0.027
```




```
from sklearn.pipeline import make_pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from lime.lime_text import LimeTextExplainer

# Sample data
texts = [
    "I love this product",
    "This is terrible",
    "Amazing experience",
    "Worst service ever"
]
```

```
labels = [1, 0, 1, 0]

# Model pipeline
model = make_pipeline(
    TfidfVectorizer(),
    LogisticRegression()
)

model.fit(texts, labels)

# LIME Explainer
explainer = LimeTextExplainer(class_names=["Negative", "Positive"])

# Test sentence
text = "This product is amazing but expensive"

# Explain
exp = explainer.explain_instance(
    text,
    model.predict_proba,
    num_features=6
)

print(exp.as_list())
exp.show_in_notebook()
```

```
[(np.str_('is'), -0.05230564217892005), (np.str_('amazing'), 0.044146549601781715), (np.str_('product'), 0.040061593128668666),
```




```
!pip install lime scikit-image tensorflow matplotlib
```

Show hidden output

```
from google.colab import files
uploaded = files.upload()
```

Choose Files download.jpg

download.jpg(image/jpeg) - 8423 bytes, last modified: 3/23/2026 - 100% done

```
Saving download.jpg to download.jpg
```

```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.applications.resnet50 import ResNet50, preprocess_input, decode_predictions
from tensorflow.keras.preprocessing import image
from lime import lime_image
from skimage.segmentation import mark_boundaries
import os

# Load pretrained deep model
model = ResNet50(weights='imagenet')

# Get uploaded file
img_path = list(uploaded.keys())[0]

# Load image
img = image.load_img(img_path, target_size=(224, 224))
img_array = image.img_to_array(img)

# Expand dimensions
img_batch = np.expand_dims(img_array, axis=0)
img_batch = preprocess_input(img_batch)

# Predict class
preds = model.predict(img_batch)
print("Prediction:", decode_predictions(preds, top=3)[0])
```

```
# DLIME Explainer
explainer = lime_image.LimeImageExplainer()

# Explain prediction
explanation = explainer.explain_instance(
    img_array.astype('double'),
    model.predict,
    top_labels=3,
    hide_color=0,
    num_samples=1000
)

# Get important regions
temp, mask = explanation.get_image_and_mask(
    explanation.top_labels[0],
    positive_only=True,
    num_features=5,
    hide_rest=False
)

# Show explanation
plt.figure(figsize=(6,6))
plt.imshow(mark_boundaries(temp / 255.0, mask))
plt.title("DLIME Explanation (Important Regions)")
plt.axis('off')
plt.show()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/resnet/resnet50_weights_tf_dim_ordering_tf_
102967424/102967424 ━━━━━━━━━━━━━━━━━━━━ 1s 0us/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 3s 3s/step
Downloading data from https://storage.googleapis.com/download.tensorflow.org/data/imagenet_class_index.json
35363/35363 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Prediction: [('n11939491', 'daisy', np.float32(0.9892396)), ('n02206856', 'bee', np.float32(0.0034935172)), ('n02281406', 'sulp
100%                                      1000/1000 [04:11<00:00,  3.03it/s]
1/1 ━━━━━━━━━━━━━━━━━━━━ 4s 4s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 3s 3s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
```

```
!pip install lime scikit-learn pandas matplotlib
```

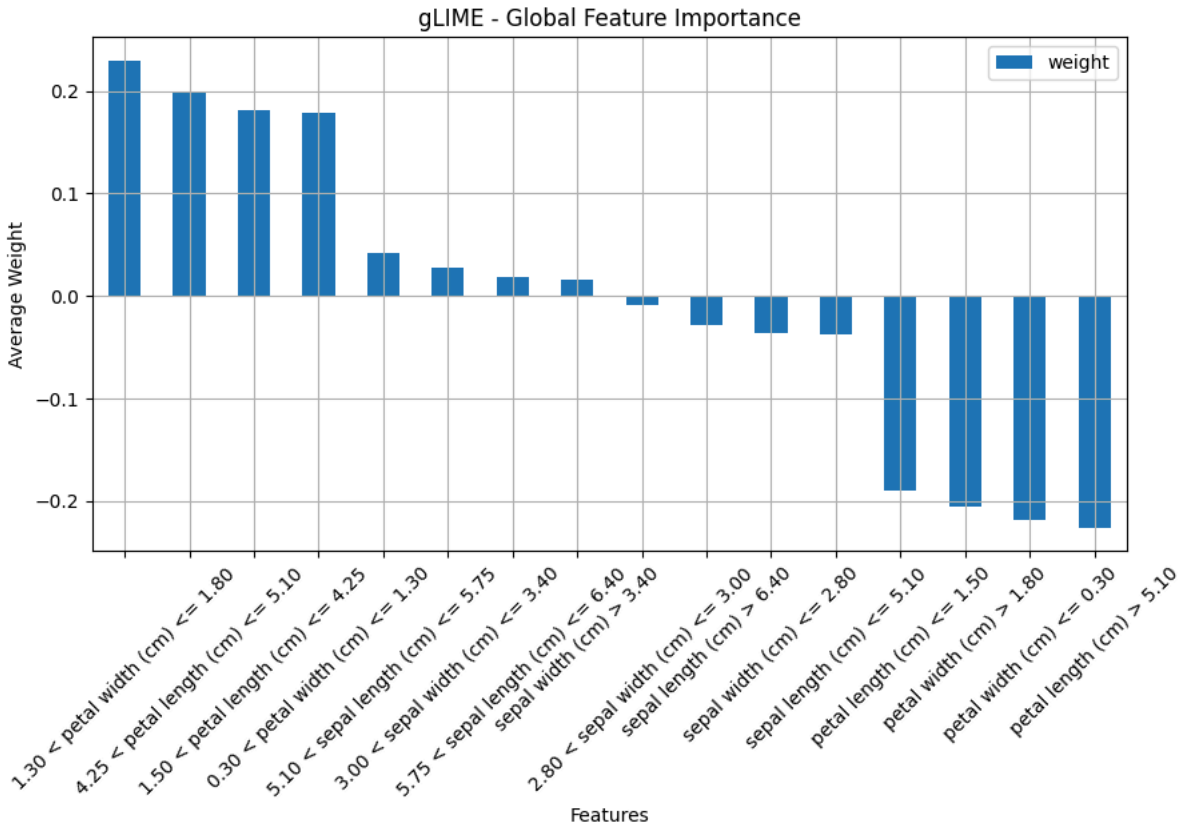
Show hidden output

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from lime.lime_tabular import LimeTabularExplainer

# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

feature_names = iris.feature_names
class_names = iris.target_names

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train model
model = RandomForestClassifier()
```

```
model.fit(X_train, y_train)

# Create LIME explainer
explainer = LimeTabularExplainer(
    X_train,
    feature_names=feature_names,
    class_names=class_names,
    mode='classification'
)

# 🔥 gLIME: Collect explanations
all_explanations = []

num_samples = 30  # number of points to explain

for i in range(num_samples):
    exp = explainer.explain_instance(
        X_test[i],
        model.predict_proba
    )

    for feature, weight in exp.as_list():
        all_explanations.append((feature, weight))

# Convert to DataFrame
df = pd.DataFrame(all_explanations, columns=["feature", "weight"])

# Aggregate (GLOBAL IMPORTANCE)
global_importance = df.groupby("feature").mean().sort_values(by="weight", ascending=False)

print("🌍 Global Feature Importance:\n")
print(global_importance)
# Plot global importance
global_importance.plot(kind='bar', figsize=(10,5))

plt.title("gLIME - Global Feature Importance")
plt.ylabel("Average Weight")
plt.xlabel("Features")
plt.xticks(rotation=45)
plt.grid()
plt.show()
```

```
Global Feature Importance:

                                    weight
feature
1.30 < petal width (cm) <= 1.80   0.230063
4.25 < petal length (cm) <= 5.10  0.200167
1.50 < petal length (cm) <= 4.25  0.181391
0.30 < petal width (cm) <= 1.30   0.178240
5.10 < sepal length (cm) <= 5.75  0.041503
3.00 < sepal width (cm) <= 3.40   0.027614
5.75 < sepal length (cm) <= 6.40  0.018639
sepal width (cm) > 3.40           0.015526
2.80 < sepal width (cm) <= 3.00  -0.008841
sepal length (cm) > 6.40         -0.028948
sepal width (cm) <= 2.80         -0.035624
sepal length (cm) <= 5.10        -0.038068
petal length (cm) <= 1.50        -0.189482
petal width (cm) > 1.80          -0.205792
petal width (cm) <= 0.30         -0.219065
petal length (cm) > 5.10         -0.226014
```




```
!pip install lime datasets scikit-learn pandas
```

Show hidden output

```
from datasets import load_dataset
import pandas as pd

# Load dataset
dataset = load_dataset("amazon_polarity")

# Convert to pandas
df = pd.DataFrame(dataset['train'][:5000])

# Rename columns
df = df.rename(columns={"content": "Text", "label": "label"})

df.head()
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster d
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to en
README.md:       6.81k/? [00:00<00:00, 153kB/s]
amazon_polarity/train-00000-of-00004.par(…): 100%  260M/260M [00:08<00:00, 112MB/s]
amazon_polarity/train-00001-of-00004.par(…): 100%  258M/258M [00:04<00:00, 144MB/s]
amazon_polarity/train-00002-of-00004.par(…): 100%  255M/255M [00:02<00:00, 273MB/s]
amazon_polarity/train-00003-of-00004.par(…): 100%  254M/254M [00:05<00:00, 48.3MB/s]
amazon_polarity/test-00000-of-00001.parq(…): 100%  117M/117M [00:02<00:00, 99.7MB/s]
Generating train split: 100%  3600000/3600000 [00:39<00:00, 207914.81 examples/s]
Generating test split: 100%  400000/400000 [00:02<00:00, 244229.46 examples/s]
```

	label	title	Text
0	1	Stuning even for the non-gamer	This sound track was beautiful! It paints the ...
1	1	The best soundtrack ever to anything.	I'm reading a lot of reviews saying that this ...
2	1	Amazing!	This soundtrack is my favorite music of all ti...
3	1	Excellent Soundtrack	I truly like this soundtrack and I enjoy video...
4	1	Remember, Pull Your Jaw Off The Floor After He...	If you've played the game, you know how divine...

Next steps: Generate code with df New interactive sheet

```
import numpy as np

# Convert labels
def convert_label(x):
    if x == 0:
        return 0  # Negative
    else:
        return 2  # Positive

df['label'] = df['label'].apply(convert_label)

# Add Neutral class artificially (20%)
neutral_indices = df.sample(frac=0.2, random_state=42).index
df.loc[neutral_indices, 'label'] = 1

# Check distribution
print(df['label'].value_counts())
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    df['Text'], df['label'], test_size=0.2, random_state=42
)
```

```
label
0    2187
2    1813
1    1000
Name: count, dtype: int64
```

```
from sklearn.pipeline import make_pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression

model = make_pipeline(
    TfidfVectorizer(stop_words='english', max_features=10000, ngram_range=(1,2)),
    LogisticRegression(max_iter=200)
)

model.fit(X_train, y_train)
```